SHoEbox: learnable recall workspace

November 6, 2026 | Configurable item-or-null slots with task-trained feedback

The recall workspace is integrated and retained for broader training. It keeps retrieved items in separate roles through the input maps at the expert composer boundary. Slot count, selection, empty slots, repeated-owner penalties and feedback are all explicit controls. Code comments mark role specialization and feedback as potentially training-threshold-dependent features for later ablation.

How the pieces work together

1. Ordinary byte context and the previous composer output form a request. Each slot has its own learned query and null preference.

2. Each slot selects one valid publication owner or remains empty. A learned coverage penalty discourages repeatedly selecting the same owner while permitting useful reuse.

3. Slot-specific input maps preserve item roles until they enter the sparse expert composer. A pooled control deliberately erases those roles while keeping the same parameters and selection work.

4. The composer output becomes feedback for the next request. Expected read usage also feeds the publication policy; task loss trains both feedback paths.

The stored publications remain the memory cells. Recall slots are current views, with no second replay store. Reads precede the current event's publication commits. The only extra carried state at a fixed owner count is the previous composer output: 320 bytes per stream at width 80 in float32.

Configurable controls

Control	Purpose
`--recall-slots N`	Available item-or-null ports; zero preserves the legacy reader
`--recall-layout distinct/pooled`	Preserve or erase item roles before the input maps
`--recall-feedback` / `--no-recall-feedback`	Enable or disable request and publication feedback
`--recall-hard` / `--no-recall-hard`	Hard selections with soft surrogate gradients, or soft reads
Existing state/payload controls	Combine with producer-state holds and delayed payload credit

Live training reaches all new controls. Version-owned delayed credit still reaches earlier publication admission through the selected reads. Request-network and recurrent feedback derivatives span the live training window; their older derivatives are not reconstructed.

Seven-slot integration

A fresh 600,354-parameter model completed 32 updates across acquisition, bridge and target stages, using seven slots over nine owners. State holds and delayed payload credit were enabled together. It processed 4,096 bytes and executed 491,520 experts.

At the final target validation it used **5.73 active slots covering 4.13 distinct owners per event**, on average. All three feedback projections learned nonzero weights. Seven is an available capacity; the model decides how much to occupy.

Matched continuation check

Two arms continued from the same 600-update parent for 160 additional updates, with a fresh optimizer and runtime state. Both trained every weight, used 564,519 parameters, processed 81,920 bytes and executed 4,915,200 experts. The two-slot addition contributes 15,444 parameters, 2.81% above the parent.

Exact-answer NLL in nats per answer byte; lower is better. Selection uses all-answer validation NLL. Each held-out split contains 16 documents; long-delay evaluation triples the configured delays.

Layout	Selected test	Final test	Selected long	Final long
Distinct	1.1609	1.2560	1.1579	1.2159
Pooled	1.1382	1.1374	1.1301	1.1206

The pooled control has lower held-out NLL in this short continuation. The distinct-slot mechanism remains in the development track for broader training. The distinct input maps learned different weights (combined deviation from their mean: 0.591); pooled maps stayed identical.

The distinct arm can learn different transformations for different slots; the pooled arm cannot distinguish which selected item occupied each role. For example, separate role maps can form A-B; averaging A and B first erases that ordering. The integration test verifies this capability directly. This representational benefit is the reason to retain the mechanism through longer training.

Do the learned controls affect answers?

Each intervention starts before the whole query and preserves the same pre-query runtime state. Positive NLL change means intact behavior helped. The pooling intervention uses the learned input maps after averaging selected items; feedback-off disables both feedback paths.

Distinct arm	Pool roles	Disable feedback	Shuffle publications
Selected test	-0.00047	-0.00256	-0.01123
Final test	+0.00051	+0.01048	-0.00148
Selected long	-0.01072	-0.01261	+0.00069
Final long	-0.00239	-0.01378	+0.00383

All eight checkpoint/split audits reproduced the live answers, with zero argmax changes and maximum logit drift 4.77e-06. Publication shuffles preserve pre-query composer and recall feedback state; normal query-time updates continue.

Verification and next integration

All 520 checks across five suites passed, along with the live-output example. Eight new checks cover capacity/null behavior, an analytic straight-through derivative, distinct roles, legacy initialization, causal read ordering, gradients, delayed credit, row resets, query forks, seven-slot execution and exact optimizer/state/stream resume across curriculum transitions.

Next, reuse this workspace as the ordinary region-to-region recall interface and extend training on mixed language and memory tasks. Carry the learned controls forward; use broader ablations to decide which mechanisms earn their cost after specialization.

Reproduce the paired continuation:

```
python scripts/run_recall.py --parent /path/to/parent.pt \
  --out artifacts/recall/study
```

The companion archive includes source, the common parent, selected/final checkpoints, the seven-slot stage checkpoints, optimizer and runtime state, per-slot query traces, audit results, validation logs and SHA-256 file hashes.